

Lecture 12: Files and Manual Serialization

Mutable collections, persistence, and parsing

Agenda

1. Mutable collections in OOP designs
2. Working with files (JVM Kotlin)
3. Manual serialisation and deserialisation

Collections as object state

Domain objects often own collections:

- `Order` holds `mutableListOf` line items
- Prefer exposing *read-only views* or controlled mutators
- Remember: `val list = mutableListOf(...)` — the reference is fixed; contents change

Encapsulated mutable list

```
1 class Team(val name: String) {  
2     private val members = mutableListOf<String>()  
3     fun add(member: String) { members += member }  
4     fun roster(): List<String> = members.toList()  
5 }
```

Outside code cannot clear the list behind your back.

Common mutable APIs

```
1 val xs = mutableListOf(1, 2)
2 xs.add(3)
3 xs[0] = 10
4 xs.removeAt(1)
5 val map = mutableMapOf("a" to 1)
6 map["b"] = 2
```

Choose list vs map vs set based on access pattern.

Quiz 1 — val + mutable

Quiz

Does this compile? `val xs = mutableListOf(1); xs.add(2)`

1. No — `val` freezes the list contents
2. Yes — `val` blocks reassignment of `xs`, not mutation of the list
3. Only with `var xs`
4. Only inside a class

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. Yes — `val` prevents `xs = ...`, not `xs.add(...)`.

Typical teaching stack:

- `java.io.File`
- Kotlin helpers: `readText()`, `writeText()`, `appendText()`
- `use { }` for streams/readers — closes resources safely

Paths are environment-specific; demos use a relative filename.

Write and read text

```
1 import java.io.File
2
3 fun main() {
4     val f = File("notes.txt")
5     f.writeText("line1\nline2\n")
6     val text = f.readText()
7     println(text)
8     f.appendText("line3\n")
9 }
```

Line-oriented processing

```
1 import java.io.File
2
3 fun main() {
4     val lines = File("notes.txt").readLines()
5     for (line in lines) {
6         if (line.isNotBlank()) println(line)
7     }
8 }
```

Combine with `split` for simple tabular formats next.

Practical cautions

- Always think about missing files and empty content
- Prefer UTF-8 text for portable teaching examples
- Do not hard-code absolute machine-specific paths in submissions

Quiz

Why prefer `bufferedReader().use { ... }` over leaving a reader open?

1. It makes the file read-only forever
2. `use` closes the resource even if an exception occurs
3. It enables smart casts on `File`
4. It is required to call `split`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. `use` is Kotlin's idiom for try-with-resources-style closing.

What “manual” means here

Serialisation: object $\rightarrow$ byte/text form for storage or transfer.

Deserialisation: text/bytes $\rightarrow$ object. We do *not* require a JSON library: design a simple textual protocol and parse it with `split`, loops, and constructors.

Domain type

```
1 class Student(val id: Int, val name: String, val gpa: Double) {  
2     fun serialize(): String = "$id|$name|$gpa"  
3     companion object {  
4         fun deserialize(line: String): Student {  
5             val p = line.split("|")  
6             require(p.size == 3) { "bad record" }  
7             return Student(p[0].toInt(), p[1], p[2].toDouble())  
8         }  
9     }  
10 }
```

Round-trip via file

```
1 import java.io.File
2
3 fun main() {
4     val s = Student(7, "Ada", 3.9)
5     File("students.txt").writeText(s.serialize() + "\n")
6     val line = File("students.txt").readLines().first()
7     val loaded = Student.deserialize(line)
8     println(loaded.name)    // Ada
9 }
```

Many records

```
1 fun saveAll(path: String, items: List<Student>) {  
2     File(path).writeText(items.joinToString("\n") { it.serialize()  
3     })  
4 }  
5 fun loadAll(path: String): List<Student> =  
6     File(path).readLines()  
7     .filter { it.isNotBlank() }  
8     .map { Student.deserialize(it) }
```

Protocol design tips

- Choose a delimiter that will not appear in data (or escape it)
- Validate token count and types (require)
- Version your format if the class gains fields later
- Manual formats teach parsing; production often uses JSON/protobuf libraries

Try this

Extend the format with an `enum` class major and update `ser/deser`.

Quiz 3 — Deserialisation

Quiz

Given format `id|name|gpa`, what should `deserialize("1|Ada")` do in a careful implementation?

1. Invent `gpa = 0.0`
2. Silently ignore the line
3. Fail validation (e.g. throw via `require`) because the arity is wrong
4. Return `null` without any check API

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. Fail validation — wrong number of fields.

Manual deserialisation must reject malformed input rather than guess.

Quiz 4 — Combining topics

Quiz

You load students into a `mutableListOf`. Which approach best preserves encapsulation?

1. Public var `students` list mutated by everyone
2. Private mutable list + public methods `add/all(): List`
3. Store only a single `String` forever
4. Make `Student` an interface with no data

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. Private mutable storage with a controlled public API.

Summary

- Mutable collections often live behind encapsulating objects
- `File` + `readText`/`writeText`/`readLines` for simple I/O
- Manual ser/deser: explicit format, `split`, validation, round-trips

Course checkpoint

You can now model types with classes, construct them carefully, use inheritance and polymorphism, apply enums and visibility, define abstract types/interfaces, and persist objects via simple file formats.